

Sardine Freshness Classification: An In-Depth Exploration of Model Dynamics

Date : 19-Feb-2024

1. Introduction :

This project aims to develop an effective system for classifying sardine fish images into good and bad categories based on their freshness. The classification involves the use of machine learning and convolutional neural network (CNN) models, with a focus on optimizing accuracy, precision, and recall rates. The report details the evolution of the project through different versions, addressing challenges, model adjustments, and performance improvements. Additionally, discrepancies in data processing and backend changes are highlighted, providing a comprehensive overview of the project's development.

2. High-Level Tech Stack:

Common Image processing libraries, Frameworks and Development platforms used for all the freshness classification models :

- **Deep Learning Framework:** Tensorflow with Keras, Pytorch
 - **Programming:** Python
 - **Image processing libraries:** OpenCV, Pillow, Ultralytics, Scikit-Image
 - **Data visualization and Plotting:** Matplotlib, Seaborn
 - **Development platform:** Jupyter notebook, Spyder
-

3. Project Phases:

3.1 Version 1 (06-Sep-23 to 20-Sep-23)

➤ **Model Architecture** **Freshness Model Version 1** :

In the initial version, a machine learning model employing Support Vector Machine (SVM) was utilized. The feature vector for classification was derived from the saturation channel histogram of the HSV image, as it was observed that saturation differences could be indicative of fish freshness. The model's input consisted of cropped images, obtained through a pre-existing fish detection model, categorized into 'Good' and 'Bad' classes based on freshness.

➤ **Model Selection** :

Various machine learning models were experimented with during the development, but SVM consistently outperformed others in terms of classification accuracy. The decision to use SVM was based on its ability to effectively capture the nuances in saturation features for distinguishing between good and bad sardines.

➤ **Additional Training Attempt** :

In an attempt to enhance model performance, a specialized model was trained using bad data labeled with "softness" as the reason. This approach involved experimenting with multiple machine learning models, but unfortunately, satisfactory results were not achieved. The challenges in accurately classifying soft bad images persisted despite this focused training effort.

➤ **Training Data** :

The training dataset comprised 500 "clear" images for each class ('Good' and 'Bad'). This dataset selection aimed to provide a diverse set of

images for the model to effectively capture the nuances associated with both good and bad sardines.

➤ **Preprocessing Steps :**

- Use of Cropped Images

Cropped images were utilized for training and classification to mitigate the potential impact of background interference on the model learning process. The presence of diverse backgrounds in complete images was hypothesized to introduce noise and hinder the model's ability to focus on the critical features related to freshness. Additionally, the dataset included both group and single fish images. To augment the limited data, cropping group images was adopted as a strategy to generate more diverse training samples.

- Aspect ratio

- **Reasoning for Maintaining Aspect Ratio**

Maintaining aspect ratio was crucial to prevent distortion and ensure that the model learned features consistently across images. It aimed to preserve the original proportions of the sardines in the dataset, allowing the model to better capture and generalize to the variations in fish appearance. This decision was driven by the assumption that preserving the aspect ratio would contribute to more effective learning of relevant features related to freshness.

- **Decision Against Padding**

Unlike the traditional approach of using padding to maintain aspect ratio, a decision was made to forgo padding and instead adjust the orientation. This choice was driven by several considerations:

- *Mitigating color interference :*

Another factor contributing to the decision against padding was the risk of introducing confusion with colors present in the fish. Padding is typically done with complete black or complete white, which are colors also found in the fish. This could potentially introduce misleading information(noise) to the model and hinder the model's ability to differentiate between actual fish features and artificially added padding.

- *Simplification through Horizontal Orientation :*

To address the challenges associated with aspect ratio maintenance and padding, a decision was made to maintain a consistent horizontal orientation instead. This involved rotating vertically oriented images to a horizontal orientation without the use of padding. This adjustment aimed to ensure that the orientation of the images was consistent, simplifying the learning process for the model.

- *Small and Diverse Dataset Consideration :*

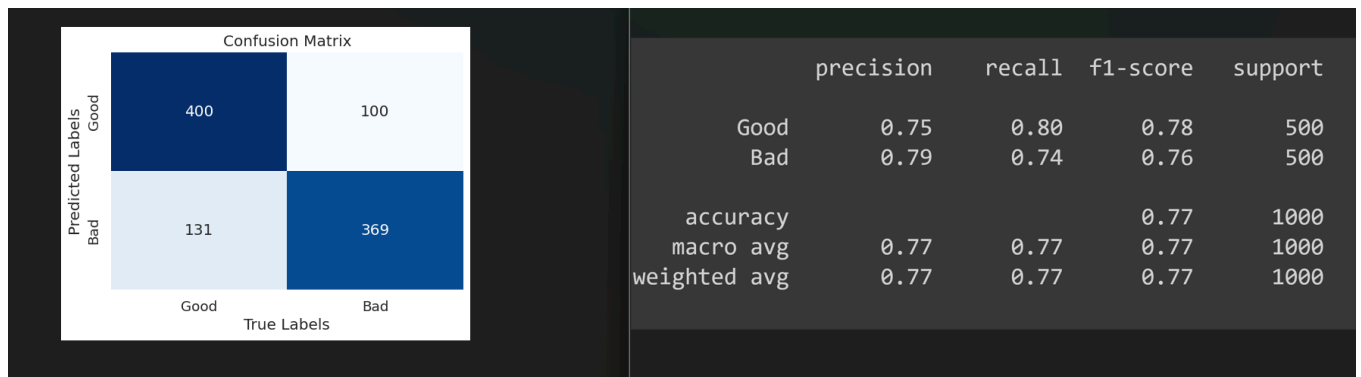
Given the small and diverse nature of the dataset, maintaining aspect ratio was deemed essential to prevent confusion in the model. The diversity within the small dataset was expected to pose challenges for the model, and maintaining aspect ratio aimed to provide a level of consistency that could aid the learning process.

- Resizing

- All images were resized to a target size of (280, 180), with the width intentionally kept greater than the height for simplicity and to preserve the horizontally oriented nature of the majority of images.
- This resizing step aimed to create a consistent input size for the SVM model, facilitating efficient training and prediction.
- The target size was selected to strike a balance between feature vector size and compatibility with commonly used pre-trained CNN models which is 224x224.

➤ **Initial test results :**

- The test dataset includes 500 images for each class.
- The overall accuracy is **77%** with balanced precision and recall between both the classes.



➤ **App testing results :**

- The overall accuracy is **70%** with notable recall and precision for the "Good" class.

V1 Sardine(06-Sep-23 to 20-Sep-23)

Actual Values			Classification Metrics Report				
Predicted Values			Precision	Recall	F1-Score	Support	
	Good	Bad					
	Good	608169	0.78	0.83	0.81	730	
			0.34	0.27	0.31	233	
	Bad	12264			0.70	963	
Accuracy			0.56	0.55	0.56	963	
Macro Avg			0.68	0.70	0.69	963	

➤ **Test Link :** [test sardine.ipynb](#)

3.1.2 Version 2 (21-Sep-23 to 22-Oct-23)

- The cuts and damages model was integrated into the sardine freshness classification project to identify and account for physical damage in sardine images, influencing their overall classification.
- Included a cuts and damages model into the system from 21-Sep-23 to enhance the classification capabilities.
- System Functionality: If the cuts and damages model predicts the presence of damage in an input image, the system immediately categorizes it as 'Bad' without further processing by the freshness classification model.

V2 Sardine(21-Sep-23 to 22-Oct-23)

Actual Values			Classification Metrics Report					
Predicted Values	Good Bad			Precision	Recall	F1-Score	Support	
	Good	692	455	Good fish	0.60	0.85	0.71	813
				Bad fish	0.55	0.24	0.34	602
	Bad	121	147	Accuracy			0.59	1415
				Macro Avg	0.58	0.55	0.52	1415
			Weighted Avg	0.58	0.59	0.55	1415	

3.1.3 Version 3 (23-Oct-23 to 19-Nov-23)

- Further refinements were made for better accuracy in identifying cuts and damages by updating the common cuts and damages model on 23-Oct-23.

V3 Sardine(23-Oct-23 to 19-Nov-23)

Actual Values			Classification Metrics Report					
Predicted Values		Good	Bad		Precision	Recall	F1-Score	Support
	Good	12	157	Good fish	0.07	0.60	0.13	20
				Bad fish	0.91	0.35	0.50	241
	Bad	8	84	Accuracy			0.37	261
				Macro Avg	0.49	0.47	0.32	261
				Weighted Avg	0.85	0.37	0.48	261

3.2 Version 4 (20-Nov-23 to 28-Nov-23)

➤ **Transition to CNN Models: Freshness Model Version 2 Evolution**

Transition Date: Transitioned from SVM to Convolutional Neural Networks (CNNs) for freshness classification in Version 2, starting from 20-Nov-23.

➤ **Dataset Expansion and Characteristics**

- Dataset for CNN Models: Expanded the dataset to include **797** bad and **795** good segmented images with a **black background**.
- Characteristics: Utilized **segmented** RGB images for training, maintaining the same pre-processing methods and target size (280x180x3) from Version 1.
- Using segmented single fish images with a black background for training models for better color information.
- Padding to maintain aspect ratio was observed to lead to information loss, particularly when the segmented fish became smaller than the background while maintaining the aspect ratio in some images. This resulted in a reduction in the amount of fish-related information available to the model. So maintaining a consistent horizontal orientation instead.

➤ **Experimental Approaches**

- Experiment Focus: Conducted five experiments to identify the most suitable combination of custom metrics for model training, primarily tracking accuracy and recall rate for the 'Good' class with MobileNetV2 and DenseNet201 as base models for transfer learning.
- Combinations Tested:
1. ['accuracy', 'macro_avg_precision', 'recall_1', 'recall_0', 'f1_score_0']
(Tracking accuracy)

2. ['accuracy', 'macro_avg_precision', 'recall_1', 'recall_0', 'f1_score_0']
(Tracking recall_1)

3. ['recall_1'] (Tracking recall_1)

4. ['recall_1', 'recall_0'] (Tracking recall_1)

5. ['recall_1', 'precision_0'] (Tracking recall_1)

○ Observations:

- Overfitting: Noted a potential overfitting issue as the accuracy reached up to 80% in the 3rd or 4th epoch.
- But the results for the 1st and 3rd combination gave better results with an accuracy of 75% and 65% accuracy for the good class and 39% and 57% for the bad class.

➤ **Model Architecture**

- Model with **DenseNet201** as base model performed better than MobileNetV2 with custom metrics recall good and precision bad.
- Additionally, I adjusted the number of dropout layers to optimize the model's performance.
- Used Sparse categorical cross entropy loss and Softmax activation function.

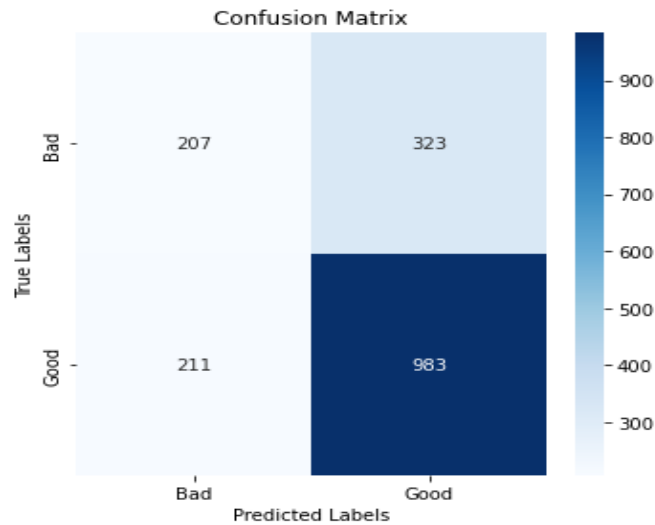
```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
densenet201 (Functional)	(None, 5, 8, 1920)	18321984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1920)	0
dense (Dense)	(None, 40)	76840
dense_1 (Dense)	(None, 50)	2050
dropout (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 50)	2550
dropout_1 (Dropout)	(None, 50)	0
dense_3 (Dense)	(None, 2)	102

```
=====  
Total params: 18,403,526  
Trainable params: 81,542  
Non-trainable params: 18,321,984
```

➤ **Initial test results**

- The results indicate a 70% recall for the positive class (good) and a 40% recall for the negative class (bad).



➤ **App testing results :**

- The overall accuracy was too low(**51%**) because we were still getting cropped images instead of segmented in the backend.

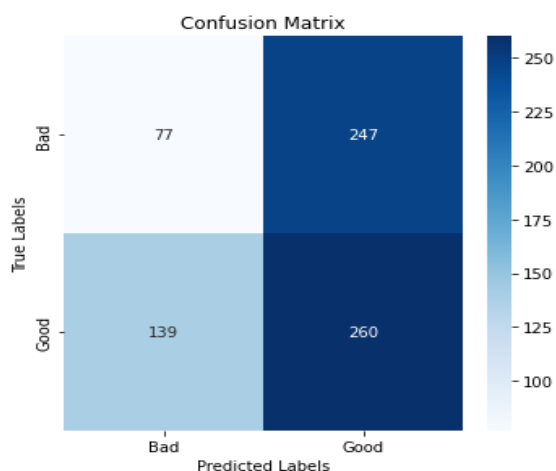
V4 Sardine(20-Nov-23 to 28-Nov-23)

Actual Values		Classification Metrics Report			
Predicted Values		Good	Bad		
	Good	43	52	Precision	Recall
	Bad	14	26	F1-Score	Support
Good fish		0.45	0.75	0.57	57
Bad fish		0.65	0.33	0.44	78
Accuracy				0.51	135
Macro Avg		0.55	0.54	0.50	90
Weighted Avg		0.57	0.51	0.49	90

➤ **Test link :** [🔗 Sample test sardine v2.ipynb](#)

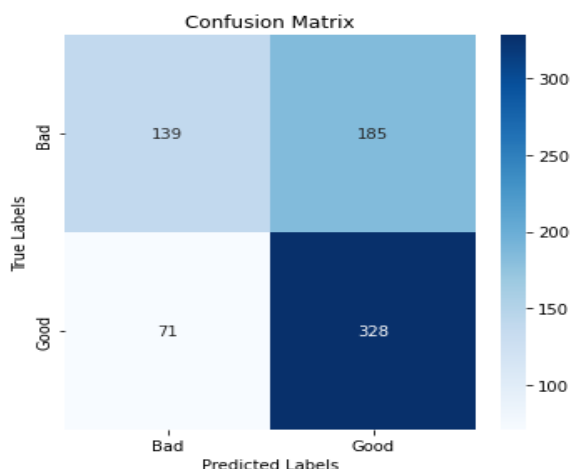
➤ **Model Comparison and Evaluation:**

- The new CNN model has already been trained on the dataset till 10-12-23. So I used a dataset after 10-12-23 for testing the model's performance.
- Conducted tests with both Version 1 and Version 2 models on different dates' data.
- Outperformed the old saturation histogram model on unseen data, exhibiting a 64% precision and 82% recall for 'Good', while the old saturation histogram model gave 51% precision and 65% recall for the 'good' class for the same dataset.



Old model results

	precision	recall	f1-score	support
Bad	0.36	0.24	0.29	324
Good	0.51	0.65	0.57	399
accuracy			0.47	723
macro avg	0.43	0.44	0.43	723
weighted avg	0.44	0.47	0.44	723



New model results

	precision	recall	f1-score	support
Bad	0.66	0.43	0.52	324
Good	0.64	0.82	0.72	399
accuracy			0.65	723
macro avg	0.65	0.63	0.62	723
weighted avg	0.65	0.65	0.63	723

3.3 Version 5 (29-Nov-23 to 30-Nov-23)

Freshness Model Version 3 Model

➤ **Model Architecture:**

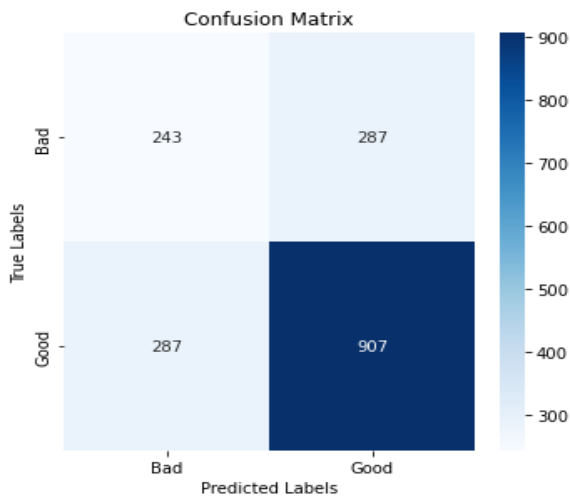
- Base Model: Densenet201
- Image Size: 280x180x3 (RGB)
- Layers:
 - Global Average Pooling
 - 40 Neurons Dense Layer
 - 50 Neurons Dense Layer
 - 20% Dropout Layer
 - 50 Neurons Dense Layer
 - 20% Dropout Layer
 - Output Layer with Softmax Activation Function
- Loss Function: Sparse Categorical Cross Entropy
- Epochs: 15

➤ **Training Details:**

- Dataset: Same as V2, consisting of segmented sardine images.
- Pre-processing: Consistent pre-processing functions and target size as V2.
- Custom Metrics: Introduced custom metrics - Recall for 'bad' class and Precision for 'good' class.

➤ **Initial test results:**

- Accuracy: Achieved an accuracy of **67%** during training.
- Recall Improvement: Noticeable improvement in recall for both 'good' and 'bad' classes with test data.



	precision	recall	f1-score	support
Bad	0.46	0.46	0.46	530
Good	0.76	0.76	0.76	1194
accuracy			0.67	1724
macro avg	0.61	0.61	0.61	1724
weighted avg	0.67	0.67	0.67	1724

➤ **App testing results:**

- The overall accuracy was too low(**48%**) because we were still getting cropped images instead of segmented in the backend.
- Precision for the 'bad' class and recall for the 'good' class were notably high.

V5 Sardine(29-Nov-23 to 30-Nov-23)

Actual Values

Predicted Values	Actual Values	
	Good	Bad
Good	57	0
Bad	63	2

Classification Metrics Report

	Precision	Recall	F1-Score	Support
Good fish	1.00	0.47	0.64	120
Bad fish	0.03	1.00	0.06	2
Accuracy			0.48	122
Macro Avg	0.52	0.74	0.35	122
Weighted Avg	0.98	0.48	0.63	122

➤ **Test link:** [🔗 Sample test sardine v3.ipynb](#)

3.3.1 Version 6 (05-Dec-23)

- **Model-Backend Mismatch:** Discrepancies between local testing and app testing data were observed, prompting investigation.
- Backend Image Issue: Despite training on segmented data, the backend received cropped images, impacting model performance.
- Code updated promptly to address the issue on Dec 5.

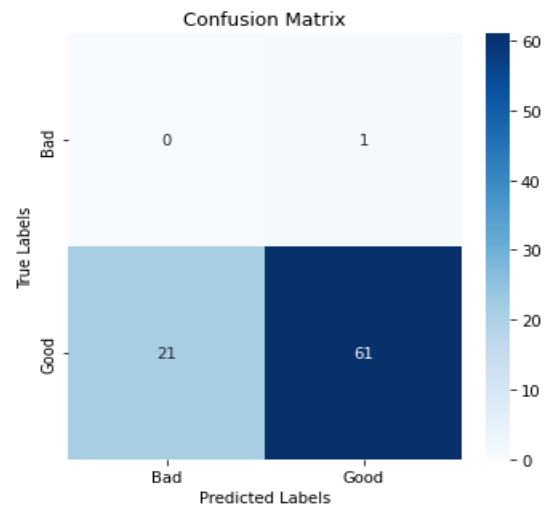
V6 Sardine(05-Dec-23)				
Actual Values		Classification Metrics Report		
Predicted Values	Good	Bad		
	Good	Bad	Precision	Recall
	0	0	F1-Score	Support
Good	0	0	Good fish	1.00
Bad	74	4	Bad fish	0.05
			Accuracy	0.00
			Macro Avg	0.10
			Weighted Avg	0.05
				78
				78
				78

3.3.2 Version 7 (06-Dec-23)

- Reversion to **Freshness Model Version 2**
 - **Model Comparison:** I conducted tests using both models on different dates' data in my local, and the results indicate that the previous model() performed better, demonstrating notably higher precision for the good class and better overall accuracy.

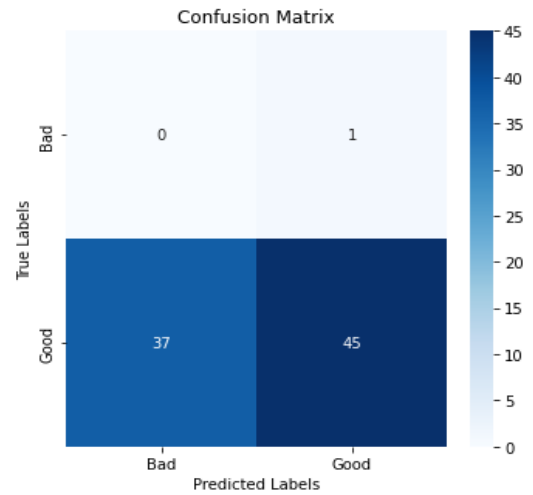
V2 Freshness model results

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	1
Good	0.98	0.74	0.85	82
accuracy			0.73	83
macro avg	0.49	0.37	0.42	83
weighted avg	0.97	0.73	0.84	83



V3 Freshness Model results

	precision	recall	f1-score	support
Bad	0.00	0.00	0.00	1
Good	0.98	0.55	0.70	82
accuracy			0.54	83
macro avg	0.49	0.27	0.35	83
weighted avg	0.97	0.54	0.69	83



➤ Backend Update Mistake:

- Error: V1 was mistakenly updated instead of V2 in the backend during app testing.
- Acknowledged the error and promptly corrected the backend by updating it with the correct V2 model.
- So app testing data on Dec 06 reflects the combined results of both V1 and V2 models. This data may not accurately represent the performance of either V1 or V2 individually.

3.3.4 Version 9 (11-Dec-23)

- **RGB to BGR Transition** : Transitioned from receiving RGB arrays to BGR arrays on Dec 11 to align with the segmentation model's training data.

V9 Sardine(11-Dec-23 to today)				
Predicted Values	Actual Values		Classification Metrics Report	
	Good	Bad		
	Good	Bad	Precision	Recall
	780	50	F1-Score	Support
Bad	60	18	Good fish	0.94
			Bad fish	0.23
			Accuracy	0.93
			Macro Avg	0.25
			Weighted Avg	0.88
Today : 19-Feb-24				908

4. Current Model:

- Freshness Model Version 2. Refer to **3.2 Version 4** for more details about the model.
- Comparing the current Freshness Model Version 2 with its predecessors reveals significant performance enhancements.

5. Performance Metrics:

- Accuracy** gives the overall correctness of the predictions.
$$\text{Accuracy} = (\text{Correct predictions}) / (\text{Total predictions})$$
- Precision** measures how many of the predicted positive cases were actually positive.

$\text{Precision} = (\text{True Positives}) / (\text{True Positives} + \text{False Positives})$

- c. **Recall** measures how many of the actual positive cases were correctly predicted.

$\text{Recall} = (\text{True Positives}) / (\text{True Positives} + \text{False Negatives})$

- d. **F1-score** is the harmonic mean of precision and recall and provides a balance between them.

$\text{F1-Score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$

6. Conclusion:

The project has undergone a transformative journey, evolving from an initial SVM model to the current CNN-based Freshness Model Version 2. While challenges such as overfitting and backend discrepancies were encountered and addressed, the project has demonstrated a substantial improvement in classification accuracy and recall rates, particularly for identifying 'Good' sardines. The decision to transition to CNNs has proven fruitful, emphasizing the importance of leveraging advanced neural network architectures for image classification tasks.

7. Future Work:

The successful implementation of the Sardine Freshness Classification project lays the groundwork for potential enhancements and expansions. Here are key areas for future work:

- **Model Exploration:** Continuous exploration of different CNN architectures and transfer learning techniques could be valuable.
- **Regular Model Retraining:** Given the dynamic nature of fish quality, establishing a system for regular model retraining with new data will be crucial to maintain accuracy and adapt to evolving conditions.
- **Dataset Size Expansion:** Increase the size of the training dataset to accommodate the growing availability of data. A larger dataset can enhance

the model's ability to generalize across diverse sardine images, improving overall performance.

- **Integration Testing:** Regularly test the model with new data collected after its integration into the app. This step ensures that the model continues to perform well in the app's operational environment and remains adaptable to evolving conditions.
- **Hyperparameter Tuning:** Conduct a thorough exploration of hyperparameter tuning to optimize the model's architecture. Experiment with different learning rates, batch sizes, and regularization techniques to enhance overall model performance.
- **Custom Loss Functions:** Investigate the impact of employing different custom loss functions in training. Experiment with loss functions tailored to specific project requirements, aiming to further improve the model's ability to capture nuanced features related to freshness.
- **Different Dataset Versions:** Explore the creation of various dataset versions, capturing different aspects of sardine freshness. This includes variations in lighting conditions, backgrounds, and the presence of other elements, ensuring the model's robustness and generalization across diverse scenarios.
- **Interpretability and Explainability:** Develop methods to interpret and explain the model's predictions. This could involve techniques such as feature importance analysis, saliency maps, or other explainability tools to provide insights into the decision-making process of the model.
- **Robustness to External Factors:** Assess the model's robustness to external factors such as variations in image quality, different camera specifications, or changes in environmental conditions. Enhance the model's adaptability to ensure consistent performance across diverse real-world scenarios.
- **Blur and Clear Image Classification:**
 - Develop a specialized model to classify images into blur and clear categories. This additional classification can provide insights into the impact of image clarity on freshness prediction.

-
- Extend the model to handle both segmented and normal images, allowing for more versatile dataset creation and evaluation.

➤ **Ongoing Performance Monitoring:**

- Establish a regular monitoring mechanism to assess the model's performance on app testing data, ensuring its effectiveness in real-world, unseen scenarios.
- Implement continuous evaluation practices to identify potential drifts in model accuracy or other metrics, prompting timely adjustments or retraining if necessary.